System Architecture

SC2006 Team 4 · Dialysis Transport Coordination System · Lab 3 Deliverable

This document describes the high-level structure of the system: its components, the connectors through which they communicate, the architectural style that governs them, and the key design decisions on persistent data, access control, external integration and design patterns.

---

1. Architecture Diagram

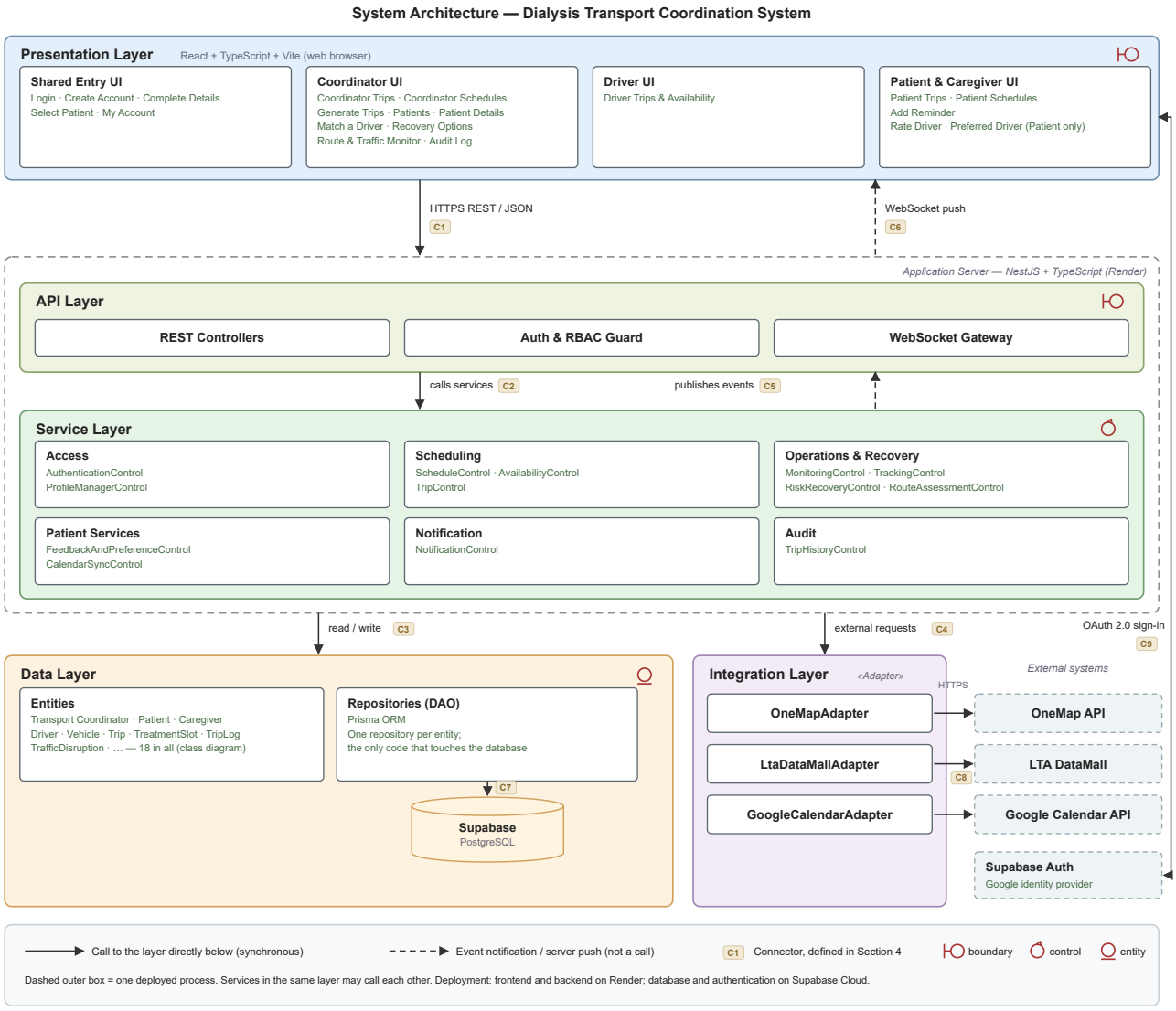


Figure 1. Layered client-server architecture. Connector labels C1–C9 are defined in Section 4. The Presentation Layer lists the main screens of each role; confirmation steps, the Schedule Form and exception states are shown in the Dialog Map.

2. Architectural Style

The system uses a **layered architecture** combined with a **client-server** style. The client is a React single-page application in the user's browser; the server is a NestJS application. Upward communication uses an **event system** with implicit invocation, described as connector C5 below.

2.1 The style, stated as a style

An architectural style is a vocabulary of component and connector types, a topology and a set of constraints. For this system these are:

- **Component types:** boundary components (screens, REST controllers, the guard and the gateway), control components (one service per area of the use case model), entity components (the domain classes and their repositories) and adapter components (one per external service).
- **Connector types:** HTTPS request-reply, local call-and-return, in-process publish-subscribe, WebSocket server push, SQL over TCP and an OAuth 2.0 sign-in redirect. These are set out in full in Section 4.
- **Topology:** four layers in a strict stack, with the Data Layer and the Integration Layer side by side beneath the Service Layer, as drawn in Figure 1.
- **Semantic constraints:** a layer may only call the layer directly below it; the browser never queries the database; every request except the sign-in redirect (C9) passes through the API Layer; the API Layer holds no business logic; services within the Service Layer may call each other; and the only upward communication is event notification, never a call, so no lower layer ever depends on a layer above it.

2.2 Design principles behind the style

- **Separation of concerns.** Presentation, business rules, persistence and external integration each sit in their own layer, so a change to one does not reach into the others.
- **Low coupling.** Layers depend on the interface of the layer below, not its implementation. Screens can change without touching scheduling rules, and the database can change without touching the services.
- **High cohesion.** Each service groups the logic for one area of the use case model, so the reason to change a service is a change to that area.
- **Modularity and abstraction.** Repositories and adapters hide storage and external APIs behind interfaces owned by the application.
- **Single responsibility.** The API Layer authenticates and routes; the Service Layer decides; the Data Layer stores. No component does two of these.
- **Open-closed.** Assignment and recovery ranking rules are separate strategy objects (Section 5.4), so a new rule is added rather than an existing service edited.
- **Traceability.** The layers match the boundary / control / entity stereotypes, so every class in the design class diagram belongs to exactly one layer and the same names are used in the code.

3. Components

Component	Responsibility	Traces to
Shared Entry UI	Login; two-step signup (Create Account, Complete Details); Account Inactive; Select Patient for Caregivers linked to more than one Patient; My Account.	UC-01; Dialog Map — Shared Entry
Coordinator UI	Coordinator Trips, Coordinator Schedules and Generate Trips, Patients and Patient Details, driver matching, recovery options, route & traffic monitor, audit log.	UC-02 to UC-05, UC-08; Dialog Map — Coordinator
Driver UI	Driver Trips and availability, Confirm Unavailability. While a trip is active, Driver Trips sends the driver's live location in the background, so the Dialog Map needs no separate state for it.	UC-06, UC-07, UC-09, UC-10; Dialog Map — Driver
Patient & Caregiver UI	Patient Trips, which shows live tracking within the page rather than on a separate screen; Patient Schedules, confirm / cancel trip, rate and preferred driver (Patient only), add reminder.	UC-03, UC-09, UC-10, UC-11, UC-11.1, UC-12, UC-13; Dialog Map — Patient & Caregiver
REST Controllers	One endpoint per user action, e.g. `POST /trips/:id/cancel`.	All use cases
Auth & RBAC Guard	Checks the login token, the user's role and, where needed, the Caregiver-Patient link before any service is called.	UC-01; NFR-5.1, NFR-5.2, NFR-5.4
WebSocket Gateway	Pushes live location, trip status and notifications to users allowed to see them.	UC-09, UC-10; NFR-6.2
Access	`AuthenticationControl`, `ProfileManagerControl`: login, profile records, Caregiver-Patient links.	UC-01, UC-02, UC-02.1
Scheduling	`ScheduleControl`, `AvailabilityControl`, `TripControl`: recurring trips, driver and vehicle assignment, availability, confirmation and cancellation.	UC-03, UC-03.1, UC-06, UC-07, UC-11, UC-11.1
Operations & Recovery	`MonitoringControl`, `TrackingControl`, `RiskRecoveryControl`, `RouteAssessmentControl`: trip status, live tracking, recovery options, route and ETA.	UC-04, UC-05, UC-05.1, UC-08, UC-09
Patient Services	`FeedbackAndPreferenceControl`, `CalendarSyncControl`: ratings, preferred driver, calendar reminders.	UC-12, UC-13
Notification	`NotificationControl`: finds the affected users and publishes notifications.	UC-10
Audit	`TripHistoryControl`: records assignments, disruptions, recoveries, cancellations and final status; serves the Audit Log.	UC-04.AF2; FR-6.4
Entities	The 18 entity classes of the design class diagram, written by the team rather than generated. Each entity carries the methods from the class diagram and enforces rules about itself, e.g. a Trip can only be cancelled while upcoming.	Design class diagram
Repositories (DAO)	Load and save entities through Prisma ORM, converting between database records and entity classes; the only code that touches the database.	NFR-3.1
Supabase PostgreSQL	Relational database for all persistent data, reached only through the repositories.	NFR-3.1

Component	Responsibility	Traces to
Supabase Auth	External identity service. The browser is redirected to it, and through it to Google, to sign in (C9); it verifies the Google identity and issues login tokens.	UC-01; NFR-5.1
Integration adapters	OneMap, LTA DataMall and Google Calendar adapters; HTTPS with a 5-second timeout, OAuth 2.0 for Google Calendar.	UC-08, UC-13; NFR-2.1, NFR-7.1

4. Connectors

Shaw and Garlan define a software architecture as its components together with its **connectors** — the interactions among those components. The connectors below are labelled C1–C9 in Figure 1.

#	Between	Mechanism	Direction and timing
C1	Browser → API Layer	HTTPS request carrying JSON, with the login token in the authorisation header	Request-reply, synchronous. Every user action is one request.
C2	API Layer → Service Layer	Local method call inside the NestJS process	Call-and-return, synchronous.
C3	Service Layer → Data Layer	Local method call on a repository interface	Call-and-return, synchronous. Wrapped in a transaction where a use case writes several records.
C4	Service Layer → Integration Layer	Local method call on an adapter interface	Call-and-return, synchronous, with a 5-second timeout.
C5	Service Layer → WebSocket Gateway	In-process event publish-subscribe (implicit invocation)	One-way, asynchronous. The publisher does not know the subscribers and does not wait.
C6	WebSocket Gateway → Browser	WebSocket server push over a connection opened by the browser	One-way, asynchronous. Not a reply to a request.
C7	Repositories → Supabase PostgreSQL	SQL over TCP/TLS, generated by Prisma	Call-and-return, synchronous, across a network boundary.
C8	Integration adapters → External APIs	HTTPS over the public internet; OAuth 2.0 bearer token for Google Calendar	Request-reply, synchronous, with a 5-second timeout and a defined failure path.
C9	Browser ↔ Supabase Auth and Google	OAuth 2.0 redirect over HTTPS	Request-reply, at sign-in only. Returns a login token, which the browser then sends on C1.

Why C5 and C6 are not calls. If a service called the browser, a lower layer would depend on a higher one and the layering constraint in Section 2.1 would be broken. Instead the service publishes an event and does not know or care who receives it; the gateway subscribes and pushes. This keeps the dependency direction downward everywhere in the system, which is why these two connectors are drawn as dashed arrows.

5. Key Design Decisions

5.1 Identifying and storing persistent data

- **Persistent objects:** all 18 entity classes of the design class diagram — Transport Coordinator, Patient, Caregiver, CaregiverPatientLink, Driver, Vehicle, AvailabilityWindow, TreatmentSlot, DialysisCentre, Trip, TripLog, RouteAssessment, TrafficDisruption, RecoveryAction, Notification, DriverRating, DriverPreference and CalendarReminder.
- **Relational database:** PostgreSQL, because the data is highly connected (a trip links a patient, driver, vehicle, treatment slot and centre) and some changes must succeed together, e.g. cancelling a trip and releasing its driver.
- **Managed service:** Supabase hosts the database, so data survives application restarts and redeployments (NFR-3.1).

- **Access through repositories only:** services never write SQL. They call repositories, so a database change is contained in the Data Layer.
- **Entity classes separate from database records:** Prisma generates plain data types with fields only. Repositories convert these records into the team's own entity classes, which carry behaviour such as `Trip.updateStatus()` or `AvailabilityWindow.overlaps()`, and convert them back when saving.
- **Transactions controlled by the Service Layer:** when one use case writes several records, the service wraps all writes in a single database transaction (Prisma `$transaction`), so they either all succeed or none are applied. For example, cancelling a trip and releasing its driver and vehicle are committed together (UC-11.1).
- **Live location:** a driver's position is pushed to viewers as it arrives. Only the latest position of an active trip is stored, and it is cleared when the trip ends (NFR-6.2).

5.2 Providing access control

- **Authentication:** users sign in with their Google account (OAuth 2.0) through Supabase Auth, which verifies the Google identity and issues a login token. There is no password form. This redirect (C9) is the only exchange that does not pass through the API Layer, and it returns only a token. Every later request must carry a valid token, which the API Layer checks (NFR-5.1).
- **Role-based access control:** each account has exactly one role. The guard checks the role on every endpoint against the access matrix below (NFR-5.2, FR-1.3).
- **Ownership checks:** beyond the role, users only reach their own data. Patients see their own trips; Caregivers only Patients they are linked to (NFR-5.4); Drivers only trips assigned to them, with only the patient information needed for the trip (NFR-6.1).
- **Checked on the server:** the server connects to the database with a privileged account, so the database itself cannot tell users apart. All checks are therefore done in the API and Service Layers, and the browser never connects to the database.

Object / operation	Transport Coordinator	Driver	Patient	Caregiver
Patient records	view; update status and notes	—	edit own (My Account)	—
Caregiver-Patient links	link	—	—	—
Treatment schedules	view	—	create, edit, delete own	—
Trips	generate, assign, delete unassigned	—	—	—
Trip details	view all	view assigned	view own	view linked Patients'
Confirm / cancel upcoming trip	—	—	own	linked Patients'
Availability	view	update own; report unavailability	—	—
Recovery options	generate, approve	—	—	—
Driver live location	view active trips	send (own active trip)	view own active trip	view linked Patients' active trip
Driver rating, preferred driver	—	—	own	—

Object / operation	Transport Coordinator	Driver	Patient	Caregiver
Calendar reminders	—	—	own	linked Patients'
Audit log	view	—	—	—

5.3 Integrating external services

- Each external service has its own adapter behind an interface owned by the application, so an API change affects one adapter only (NFR-7.1).
- Routing and traffic calls time out after 5 seconds; the timeout appears as the *Service Unavailable* state in the Dialog Map instead of stale data (NFR-2.1, UC-08.EX1).
- A Google Calendar failure only affects reminders; scheduling and monitoring continue (NFR-3.2, UC-13.EX1). Calendar access needs the user's OAuth consent (NFR-5.3, UC-13.EX2), and only the trip fields needed for the reminder are sent (NFR-6.3).

5.4 Design patterns

Pattern	Where it is used	Why
Layered architecture	Presentation, API, Service and Data Layers	Separation of concerns; each layer can be replaced without disturbing the others.
Client-server	Browser single-page application and NestJS server	Business rules and security are centralised on the server.
Adapter	OneMap, LTA DataMall and Google Calendar adapters	An external API change affects one adapter only (NFR-7.1).
Data Access Object	Prisma repositories in the Data Layer	Storage details are hidden behind an interface, so services never write SQL.
Observer (publish-subscribe)	`NotificationControl` publishes events; the WebSocket Gateway subscribes and pushes them to the browsers of the affected users	Senders do not need to know who is listening. This is the implicit-invocation form of an event system.
Strategy	The rules that rank candidate assignments in `ScheduleControl` and candidate recovery actions in `RiskRecoveryControl` are separate interchangeable objects behind one interface	UC-03.1 applies mandatory constraints, then patient preferred driver, then driver continuity; UC-05 ranks recovery options. Keeping each rule as its own strategy means a new or reordered rule is added without editing the service — the Open-Closed Principle.

6. Meeting the Non-Functional Requirements

The architecture exists to satisfy the quality attributes, not only the functions. The table below states which architectural mechanism delivers each one.

Quality attribute	Requirement	How the architecture delivers it
Security	NFR-5.1, NFR-5.2, NFR-5.4	All checks run on the server. The Auth & RBAC Guard sits in front of every service call, and the browser never connects to the database, so there is a single place to enforce authentication, role and ownership.
Reliability	NFR-3.1, NFR-3.2	Persistent state is held in a managed PostgreSQL instance rather than in the application process, so it survives restarts and redeployments. A Google Calendar failure is contained in one adapter and does not stop scheduling or monitoring.
Robustness	NFR-2.1	External calls time out after 5 seconds and surface as the *Service Unavailable* state in the Dialog Map, so the coordinator is shown a recoverable failure instead of stale or missing data.
Efficiency	NFR-2.1, NFR-6.2	Live location and trip status are pushed over C5 and C6 rather than polled, so the browser does not repeatedly ask for data that has not changed.
Maintainability	NFR-7.1	One adapter per external service, and one repository per entity, so an external API change or a database change is contained in a single layer.
Privacy	NFR-6.1, NFR-6.2, NFR-6.3	Each push and each response is filtered by the viewer's role and link before it leaves the server. Only the latest position of an active trip is stored, and it is cleared when the trip ends.

7. Constraints

- **Languages:** TypeScript on both sides — React with Vite in the browser, NestJS on the server. One language across the stack keeps the entity definitions and their validation rules shared rather than duplicated.
- **Libraries and services:** Tailwind CSS for styling, Prisma as the ORM, Supabase for PostgreSQL and authentication, Jest, Vitest and React Testing Library for tests. These were confirmed in the Lab 3 technology-stack exercise and are unchanged from Lab 2.
- **Communications:** HTTPS for all browser-to-server and server-to-external traffic; WebSocket for server push; OAuth 2.0 for Google sign-in and Google Calendar. No other protocol is used.
- **Deployment:** frontend and backend on Render; database and authentication on Supabase Cloud. A single server process, with no microservices and no separate backend in another language — not justified by the current design at this scale.
- **Team:** the team already knows TypeScript, React, NestJS and Supabase, so the architecture deliberately avoids technologies that would need to be learned before the build starts.